Learning Planning-based Reasoning via Trajectories Collection and Process Reward Synthesizing

Fangkai Jiao[1,2] **Chengwei Qin**[1] **Zhengyuan Liu**[2]
Nancy F. Chen[1,2†] **Shafiq Joty**[3,1†]
[1]Nanyang Technological University, Singapore
[2]Institute for Infocomm Research (I^{2}R), A*STAR, Singapore
[3]Salesforce Research, USA
jiaofangkai@hotmail.com chengwei003@e.ntu.edu.sg
sjoty@salesforce.com {nfychen, liu_zhengyuan}@i2r.a-star.edu.sg

Abstract

Large Language Models (LLMs) have demonstrated significant potential in handling complex reasoning tasks through step-by-step rationale generation. However, recent studies have raised concerns regarding the hallucination and flaws in their reasoning process. Substantial efforts are being made to improve the reliability and faithfulness of the generated rationales. Some approaches model reasoning as planning, while others focus on annotating for process supervision. Nevertheless, the planning-based search process often results in high latency due to the frequent assessment of intermediate reasoning states and the extensive exploration space. Additionally, supervising the reasoning process with human annotation is costly and challenging to scale for LLM training. To address these issues, in this paper, we propose a framework to learn planning-based reasoning through Direct Preference Optimization (DPO) on collected trajectories, which are ranked according to our synthesized process rewards. Our results on challenging logical reasoning benchmarks demonstrate the effectiveness of our learning framework, showing that our 7B model can surpass the strong counterparts like GPT-3.5-Turbo. [1]

1 Introduction

Natural language reasoning has been a fundamental element in the advancement of artificial intelligence (AI), with its significant impact on a variety of applications including planning and decision making (Huang and Chang, 2023). The goal of building AI systems capable of replicating human-like reasoning remains a primary focus within the research community. Recent advancements in Large Language Models (LLMs) have showcased their ability to perform complex reasoning tasks, creating sequences of reasoning steps akin to human thought processes (Wei et al., 2022; Zhou et al., 2023b). Despite these advancements, it is also concerning that LLMs are susceptible to generating misleading rationales (Bubeck et al., 2023; Lanham et al., 2023). Such inaccuracies are particularly pronounced in complex reasoning scenarios (Yu et al., 2022; Huang and Chang, 2023; Jiao et al., 2024; Wang et al., 2024), underscoring a significant challenge.

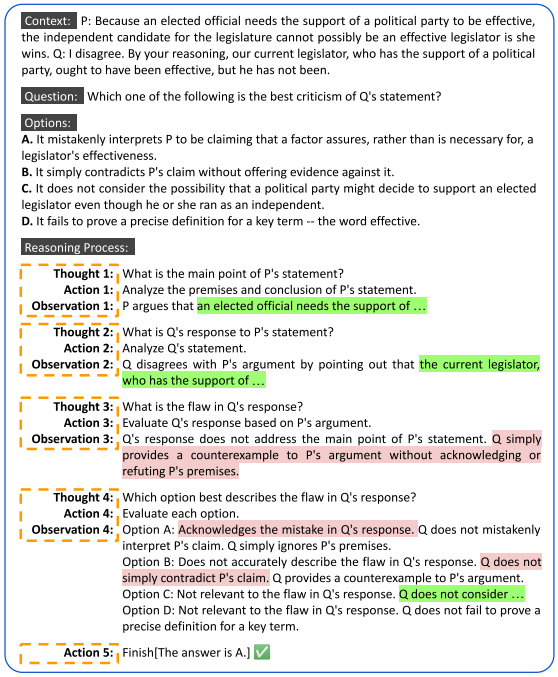


Figure 1: A solution generated by our fine-tuned model based on Llama-2-7B-chat (Touvron et al., 2023) for a logical reasoning problem in LogiQA-v2 dataset (Liu et al., 2022). It follow the ReAct (Yao et al., 2023b) format, where each step is marked with a dotted rectangle. The content highlighted in green summarizes some opinions in the context, and is omitted. The central reasoning steps pivotal to arriving at the solution are emphasized in pink. The complete reasoning process can be found in Figure 8.

Tremendous efforts have been dedicated to improve the reliability and faithfulness of generated rationales, including knowledge distillation (Hinton et al., 2015; Xu et al., 2023; Luo et al., 2023;

[†]Correspondence to: Nancy F. Chen and Shafiq Joty.
[1]Code and trajectory data are released at SparkJiao/dpo-trajectory-reasoning.

Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, pages 334–350
November 12-16, 2024 ©2024 Association for Computational Linguistics

Yue et al., 2023) and self-correction (Shinn et al., 2023). Yet, these approaches predominantly rely on LLMs for identifying errors or providing superior reasoning processes, which could be limited by their capacity. An alternative is to consider *human* process supervision (Uesato et al., 2022). For instance, Lightman et al. (2023a) propose to train a process reward model (PRM) using step-level feedbacks on model-generated solutions, which are annotated by human experts. This enables LLMs to refine their rationales based on the PRM's feedback. While human process supervision has proven effective, it often incurs higher costs compared to mere final outcome annotation as well as the automatic process annotation from a teacher LLM.

In addition to the attempts on process supervision, some research efforts have explored search-augmented reasoning for better reasoning trace by assessing the quality of future states. Hao et al. (2023) introduce a general framework of reasoning-as-planning (RAP), where the reasoning process is defined as a Markov Decision Process (MDP). Each step in the MDP comprises a *state-action* pair, whose particular implementation can vary with different application scenarios. Figure 1 illustrates this process in the context of logical reasoning using the ReAct (Yao et al., 2023b) format. At each step the agent can *Think* (optionally) and *Act*, which involves selecting a group of facts and rules to deduce a new conclusion.[2] It can optionally make an *Observation* to get an "updated view" of the *state*. During inference, each state-action pair is assigned a reward, either by an LLM or external verifier. The planning process is then steered by Monte Carlo Tree Search (MCTS) (Coulom, 2006) to maximize the expected total cumulative reward (or utility) obtained along the chosen path while effectively narrowing the search space (Figure 2(a)).

Existing RAP frameworks often assume LLMs as the world model being able to assess the quality of each reasoning step. As a result, the online planning may introduce huge latency and cost due to frequent assessments of intermediate states and the large search space. Nevertheless, we find that the core idea behind planning-based reasoning is to employ online simulation by taking few forward steps to find the optimal path, and the evaluation becomes more accurate when it has access to real outcome feedback.

In this paper, we explore *offline simulation* to synthesize process supervision. We introduce ground-truth outcome supervision that we back-propagate to intermediate states instead of relying on LLMs for process assessment. We develop a simple and effective strategy based on partial trajectory exploration. We first collect some solutions from LLMs as the seed trajectories, and then sample several intermediate reasoning states from them as the non-leaf nodes in planning. After that, the LLMs are asked to retry to complete each of them multiple times by taking the intermediate states as new starting points of reasoning. We take the number of completions that have reached the correct outcome as the estimate of expected returns for training PRM. Finally, we optimize the LLMs to learn a better policy for generating reliable rationales through Direct Preference Optimization (Rafailov et al., 2023), where the contrastive trajectory pairs are annotated by the PRM. A general comparison between our method and search-based approaches is shown in Figure 2. In a nutshell, our contribution can be summarized as follows:

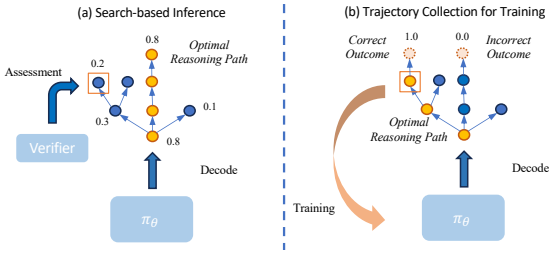


Figure 2: The overall comparison between search-based inference (a) and our trajectory collection-based offline training (b). In search-based inference, an LLM or an external verifier assesses each intermediate state and assigns a scalar value as feedback. The goal of inference is find an optimal reasoning path with maximum expected utility. In our method, the policy model will first explore multiple reasoning paths, with the process rewards calibrated by outcome supervision. And we then optimize it using DPO (Rafailov et al., 2023) to maximize the probability of the paths with higher cumulative reward.

- We propose a novel framework for synthesizing process rewards from outcome annotations, which incorporates offline simulation and trajectory collection to induce planning-based reasoning.

- We rigorously evaluate our methodology on two challenging reasoning tasks: logical reasoning and mathematical reasoning. The observed significant improvements over robust baseline models underscore the efficacy of our

[2]The notion of *action* subsumes both thinking and acting.

proposed approach.

- Through detailed analysis, we demonstrate that our method not only improves the quality and conciseness of generated rationales but also reduces the reliance on human annotations.

2 Related Work

2.1 LLMs for Reasoning

Compared with predicting only the final answer, chain-of-thought (CoT) (Wei et al., 2022) serves as a more suitable way for LLMs considering the rationale will derive more useful information to avoid potential flaws. Following this, many prompting techniques are proposed to enrich the generated rationales (Zhou et al., 2023b; Hao et al., 2023). Another group of work focuses on search-augmented reasoning, where the decoding process is guided by heuristic search algorithms, like MCTS (Coulom, 2006). Basically, each reasoning state is treated as a node in a tree or graph, and assigned with a value demonstrating the confidence or expected rewards when reaching it. And LLMs themselves often serve as the evaluator to give feedback to intermediate states (Yao et al., 2023a; Hao et al., 2023).

2.2 Improving LLMs via Sparse Feedback

Since the success of reinforcement learning from human feedback (RLHF) (Christiano et al., 2017; Ouyang et al., 2022), employing RL algorithms, like PPO (Schulman et al., 2017), to optimize LLMs from sparse feedback is becoming more important. However, PPO training often demonstrates unstable process and high resource cost. Some alternative variants are then proposed, like rejection sampling (Bai et al., 2022; Touvron et al., 2023) and direct preference modeling (DPO) (Rafailov et al., 2023). Towards the different types of feedback, Lightman et al. (2023b) and Uesato et al. (2022) propose process supervision to assess the intermediate reasoning steps. Nevertheless, collecting step-wise feedback from human experts is often time-consuming and expensive. In this paper, we propose a simple heuristic approach to estimate the process rewards of intermediate states.

Our work is concurrent to MATH-Shepherd (Wang et al., 2023). We share similar methodology for process rewards estimation, but we have focused on different reasoning tasks, optimization approaches, and evaluations. More details are discussed in Appendix D.

3 Method

3.1 Formal Definition of Natural Language Reasoning

Following Hao et al. (2023), we define the natural language reasoning task as a MDP with an action-state trajectory: $\tau = \langle s_0, a_0, \cdots, s_t, a_t, \cdots, s_T, a_T \rangle$, where a_t is the action taken at timestep t and s_{t+1} is the state that the agent observes after that. In the context of LLMs, we simplify the setting by considering that both the action and state are sampled from the policy model π_θ (an LLM), such that:

$$\begin{cases} a_t \sim \pi_\theta(a|c_t), \\ s_{t+1} \sim \pi_\theta(s|a_t, c_t), \end{cases} \tag{1}$$

where θ is the parameter of the policy model, $c_t = (s_0, a_0, \cdots, s_t)$ is the history trace. Besides, a reward model $r_t = r(a_t, s_t) \in \mathbb{R}$ is employed to assess the feasibility and desirability of each state-action pair. In this paper, we focus on the tasks with annotated final labels, where the agent will receive a positive reward when it finally reaches a correct answer:

$$r_f(\tau, y) = \begin{cases} 1, & \text{if } \tau \to y \\ 0, & \text{else} \end{cases} \tag{2}$$

where y is the ground-truth answer of a given query, and $\tau \to y$ means the trajectory entails the prediction y. Our aim is to optimize the policy for making decisions to maximize the expected rewards, which can be formulated as:

$$\underset{\theta}{\operatorname{argmax}}\, \mathbb{E}_{x,y\sim\mathcal{D},\tau'\sim\pi_\theta(\tau|x)} r_f(\tau', y), \tag{3}$$

where π_θ is the policy model parameterized by θ, $\mathcal{D} = \{x^{(i)}, y^{(i)}\}$ is the dataset where the policy model is optimized on, x is the concatenation of prompt, context, and question, and τ' is the generated reasoning process as action-state sequence.

3.2 Estimate Process Rewards via Offline Simulation

One of the main issues with LLMs is that they tend to hallucinate (Huang et al., 2023). A common illusion with multi-step reasoning is that the derived conclusion may be correct but the LLMs might reach there through unreasonable deduction

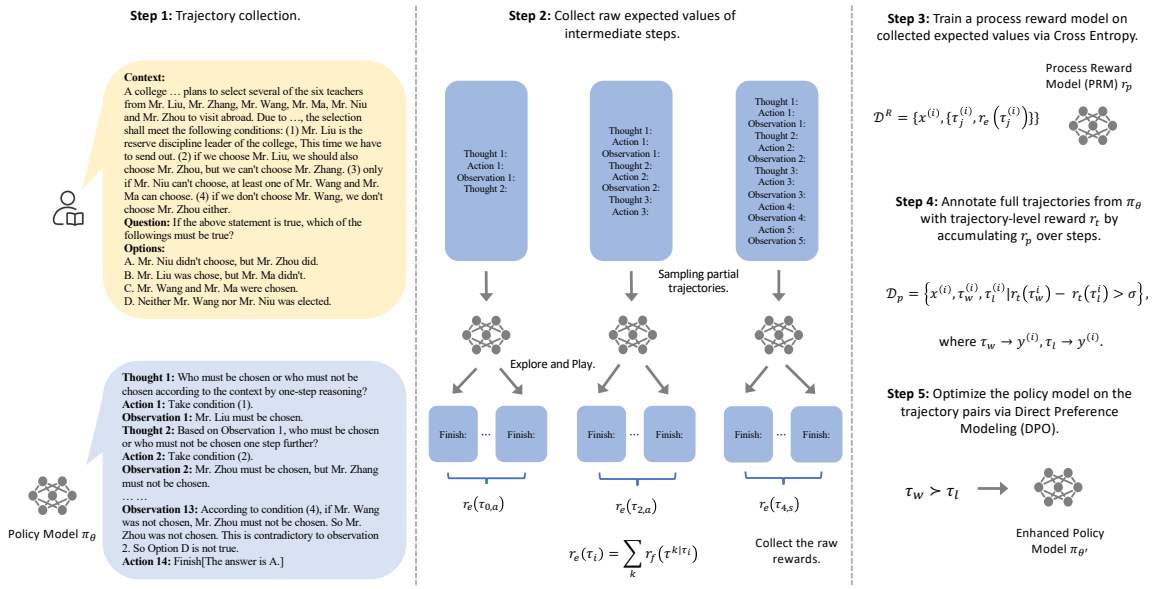


Figure 3: The overall framework of our approach. (1) Collect samples with full solution trajectories. (2) Sample intermediate reasoning states from the dataset, and ask the policy model to continuously explore based on the intermediate states. After the completed trajectory reaching the termination, we can collect the raw rewards according to the outcome supervision as the approximation of expected returns for the intermediate reasoning states. (3) A process reward model is learned from the raw rewards to alleviate the dataset noise and reduce simulation cost. (4) Collect more full trajectories and annotate them with the trained process reward model. (5) Optimize the policy model on the pairwise trajectory dataset assessed by our synthesised process rewards.

processes. To address this, we aim at introducing process supervision (Lightman et al., 2023a), which, however, is hard to obtain in most reasoning cases. We propose a simulation based method to estimate the expected value by starting from an intermediate point in a trajectory and exploring the received rewards after reaching the terminal states. The idea is based on a common observation that if an intermediate reasoning state can reach the correct answer more frequently, it has higher probability to demonstrate some important facts or evidences towards the conclusion. Specifically, given an input x and an intermediate reasoning step t, we randomly sample K trajectories starting from either action a_t or state s_t. Taking a_t as example, the estimated expected value for it is formulated as:

$$\begin{aligned} r_e(\tau_{t,a}, y) &= \sum_k^K r_f(\tau^{k|\tau_{t,a}}, y), \\ &= \sum_k^K r_f(\langle \underbrace{s_0, a_0, \cdots, a_t}_{\text{The prefix of } \tau}, \underbrace{s_{k,t}, \cdots, s_{k,T_k}}_{\text{The sampled completion.}} \rangle, y), \end{aligned} \tag{4}$$

where $\tau^{k|\tau_{t,a}}$ is the k-th completed trajectory starting from a_t, and T_k is the number of steps in the trajectory. Note that we can estimate the expected value for both action or state, since they are all managed by the policy model. For simplicity, we will discuss the method based on action.

3.3 Synthesized Process Reward Model

After collecting enough trajectories as well as the estimated expected values of intermediate steps, we can train a PRM to assign a reward to each intermediate state/action, following Lightman et al. (2023b). The motivation behind training a process reward model instead of using the collected values as the rewards includes: (1) If we assess each intermediate step to estimate the value of the complete trajectory by only heuristic simulation, similar to the weakness of MCTS, the time consumption and cost will be severe. (2) The simulation based estimation will also introduce noise, since the completion quality highly depends on the fundamental capability of the initial policy model. As a result, employing an extra reward model to approximate the expected values can be more robust and efficient than heuristic algorithms.

Specifically, following the method in Section 3.2 we obtain a reward modeling dataset:

$$\mathcal{D}^R = \{x^{(i)}, \{\tau_{j,a}^{(i)}, r_j^{(i)}\}\}, \tag{5}$$

where $r_j^{(i)} = r_e(\tau_{j,a}^{(i)}, y^{(i)})$, and j is the step. We then formulate the reward modeling process as a classification problem with K classes, and train the process reward model $f_{\text{prm}} : \mathcal{X} \times \mathcal{T} \to \mathbb{R}^K$ by

minimizing the following Cross-Entropy loss:

$$\begin{cases} \mathcal{L}_{\text{step}} = -\log p_r, \\ p = f_{\text{prm}}(x, \tau), \end{cases} \tag{6}$$

where τ is an (incomplete) trajectory and r is the corresponding estimated real reward value.

3.4 Reward Annotation and Preference Dataset Construction

After obtaining the process rewards, we can then assess a complete trajectory by accumulating them along steps. Specifically, given a complete trajectory $\tau = \langle\, s_0, a_0, s_1, a_1, \cdots, s_T, a_T \,\rangle$, the trajectory level reward is defined as the accumulated production of the process rewards assigned at each intermediate step:

$$\begin{cases} r_p(\tau) = \prod_t^T \prod_*^{\{a,s\}} \sum_{i \geq C}^K f_{\text{prm}}(\tau_{t,*})_i, \\ \tau_{t,a} = \langle\, s_0, a_0, \cdots, s_t, a_t \,\rangle, \\ \tau_{t,s} = \langle\, s_0, a_0, \cdots, s_t, \,\rangle, \end{cases} \tag{7}$$

where $*$ indicates either a or s. C is a hyper-parameter controlling the minimum amount of successful simulations so that we have enough confidence to claim the state can lead to a correct reasoning process. This is to avoid that the potential hallucinated rationales generated by the original LLMs can affect the estimation of process rewards.

Once we have the clear definition of the trajectory level reward based on the PRM, the policy model can be optimized via reinforcement learning. Considering the instability of PPO (Schulman et al., 2017) training, we choose the algorithm of Direct Preference Optimization (DPO) instead.

3.5 Direct Preference Optimization

In this section, we will first introduce the vanilla DPO approach with only outcome supervision, which also servers as an strong baseline method. Specifically, given an original data sample $(x^{(i)}, y^{(i)})$, and a group of trajectories $\mathcal{T}^{(i)} = \{\tau_0^{(i)}, \tau_1^{(i)}, \cdots, \tau_n^{(i)}\}$ sampled from the policy model taking $x^{(i)}$ as input, we can simply construct a preference dataset:

$$\mathcal{D}_{\text{o}} = \{x^{(i)}, \tau_w^{(i)}, \tau_l^{(i)}\}, \tag{8}$$

where $\tau_w^{(i)} \in \mathcal{T}^{(i)}$ is a trajectory successfully reaching the correct answer $y^{(i)}$, and $\tau_l^{(i)} \in \mathcal{T}^{(i)}$ is another trajectory with incorrect prediction. After that, we can optimize the policy model π_θ on the dataset $\mathcal{D}_{\text{o}}$ by minimizing the following loss:

$$\begin{aligned} &\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{ref}; \mathcal{D}_{\text{o}}) \\ &= -\,\mathbb{E}_{(x,\tau_w,\tau_l)\sim\mathcal{D}_{\text{o}}}\Bigg[\log\sigma\Bigg(\beta\log\frac{\pi_\theta(\tau_w|x)}{\pi_{\text{ref}}(\tau_w|x)} \\ &\quad -\beta\log\frac{\pi_\theta(\tau_l|x)}{\pi_{\text{ref}}(\tau_l|x)}\Bigg)\Bigg], \end{aligned} \tag{9}$$

where π_{ref} is the reference model initialized from the original policy model before DPO training, β is the hyper-parameter controlling the divergence between the distribution from the policy model and the reference model, τ_w is the chosen solution, and τ_l is the rejected solution.

From the definition we can find that the vanilla DPO approach only considers the pairwise relationship based on final prediction, regardless of the reliability of intermediate reasoning process. Since we have already defined a trajectory-level reward in Equation 7 involving the process rewards, we can further consider the pair-wise relationship among those trajectories with correct predictions:

$$\mathcal{D}_{\text{p}} = \{x^{(i)}, \tau_a^{(i)}, \tau_b^{(i)} | r_p(\tau_a^{(i)}) - r_p(\tau_b^{(i)}) > \sigma, \}, \tag{10}$$

where $\tau_a^{(i)}$ and $\tau_b^{(i)}$ both induce the correct prediction $y^{(i)}$, and σ is hyper-parameter representing the confidence margin. τ_a is the chosen solution and τ_b is the rejected one. And the final objective can thus be written as $\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{ref}; \mathcal{D}_{\text{o}} \cup \mathcal{D}_{\text{p}})$.

4 Experiments

4.1 Datasets

In this paper, we mainly focus on logical reasoning and mathematical reasoning. For logical reasoning, we choose ReClor (Yu et al., 2020) and LogiQA-v2 (Liu et al., 2022) for evaluation, which are two challenging and widely used logical reasoning benchmarks. Both datasets are formulated as multiple choice question answering and the statistics of the two datasets are shown in Table 3. For mathematical reasoning, we have employed the test sets of GSM8K (Cobbe et al., 2021) and MATH (Hendrycks et al., 2021) for evaluation.

4.2 Baselines

For the baseline methods, we mainly choose the following types of approaches: (1) Foundational LLMs, including Llama2-70B-Chat (Touvron et al., 2023), Mixtral-MoE-8×7B-Instruct (Jiang et al., 2024), GPT-3.5-Turbo and GPT-4-Trubo[3]; (2) Supervised Fine-tuning (SFT), where the training solutions are sampled from larger teacher models;

[3]We use *gpt-3.5-turbo-1106* and *gpt-4-0125-preview*.

	Training Set	LogiQA-v2	ReClor
GPT-3.5-Turbo	—	45.4	53.7
GPT-4-Turbo	—	70.0	—
Llama2-70B-chat	—	43.8	60.4
Mixtral-8×7B-Instruct	—	49.5	56.7
Llama2-7B-SFT	ReClor	44.5	48.8
Llama2-7B-DPO	ReClor	**47.5**	51.3
Llama2-7B-pDPO	ReClor	47.4	**53.5**
Llama2-7B-SFT	LogiQA-v2	45.5	53.4
Llama2-7B-RFT			
Outcome	LogiQA-v2	47.8	52.3
Outcome & PRM-top-1	LogiQA-v2	48.0	54.2
Outcome & PRM-top-3	LogiQA-v2	48.1	53.0
Outcome & PRM-top-5	LogiQA-v2	47.9	52.6
Llama2-7B-ReST-EM	LogiQA-v2	49.4	51.5
Iter-1	LogiQA-v2	48.7	52.8
Llama2-7B-IPO	LogiQA-v2	44.5	54.1
Llama2-7B-DPO	LogiQA-v2	53.1	60.4
Llama2-7B-pDPO	LogiQA-v2	**55.5**	**61.7**
Iter-1-DPO	LogiQA-v2	56.7	61.0
Iter-1-pDPO	LogiQA-v2	**57.3**	**61.8**
Iter-1-process PPO	LogiQA-v2	56.2	61.2
Iter-1-process GRPO	LogiQA-v2	**57.3**	**61.7**

Table 1: Experimental results on the test set of the logical reasoning benchmarks.

(3) DPO and IPO (Azar et al., 2023) methods with only outcome supervision; (4) Rejection-sampling based approaches, including Rejection-sampling based Fine-tuning (RFT) (Yuan et al., 2023) and ReST-EM (Singh et al., 2023), and (5) reinforce learning (RL) algorithms for iterative training. The details of baselines can be found in Appendix A.

4.3 Evaluation and Implementation

The evaluation of open-ended generation is difficult. Considering that most of our models are fine-tuned on fixed format, we only take the solutions with satisfying the format requirements into consideration to calculate accuracy. The detailed rules can be found in Appendix B. The implementation details can be found in Appendix C.

5 Results and Analysis

5.1 Overall Results on Logical Reasoning

The results on logical reasoning benchmarks are shown in Table 1, from where we can conclude that (1) DPO serves as a strong baseline, significantly boosting the performance of the SFT model and outperforming the other baselines. Notably, the DPO-fine-tuned model on LogiQA-v2 records an in-domain improvement of 7.0%, and an 7.6% improvement on the ReClor dataset. The one fine-tuned on ReClor also demonstrates 2.5% in-domain and 3.0% out-of-domain improvements, respectively. Besides, on LogiQA-v2, Llama2-7B-DPO can already surpass the other rejection sampling based baselines with large margins, like RFT and ReST-EM. This indicates DPO's efficacy in optimizing the policy model using outcome supervision alone. (2) pDPO surpasses the vanilla DPO that relies solely on outcome supervision. For instance, by fine-tuning on LogiQA-v2, pDPO achieves absolute improvements of 2.4% and 1.3% on LogiQA-v2 and ReClor, respectively. Through training on ReClor, pDPO also achieves 2.2% absolute in-domain improvements. Besides, pDPO trained on LogiQA-v2 outperforms the strong foundation LLMs including Mixtral and GPT-3.5 Turbo, suggesting the superiority of our synthesized process supervision. (4) The LogiQA-v2 dataset emerges as a more effective tool for learning explicit logical reasoning processes compared to ReClor. As shown in the Table, by fine-tuning on LogiQA-v2, the generalization performance of pDPO on ReClor dataset is even better than the in-domain fine-tuned models. After diving into the dataset details, we find that LogiQA-v2 comprises multiple complex logical reasoning abilities, like categorical reasoning and sufficient reasoning, while quite a few questions in ReClor require only one-step reasoning to justify the entailment of each option.

5.2 Improvements by Iterative Training

We also performed iterative training by taking Llama2-7B-pDPO trained on LogiQA-v2 as the new base model and fine-tuning it on the newly self-sampled solutions. In addition to DPO and pDPO, we have also explored the RL based approaches, including PPO (Schulman et al., 2017) and Group Relative Policy Optimization (GRPO) (Shao et al., 2024). For fair comparison with pDPO, PPO and GRPO also include both the process rewards from our PRM, and the outcome rewards derived from the ground-truth labels. The implementation details can be found in Appendix A.

From Table 1, we observe that all four approaches demonstrate consistent in-domain improvements. Notably, the pDPO approach, which utilizes synthesized process supervision, surpasses the conventional process PPO method. This improvement may be attributed to the slightly noisy nature of the synthesized process rewards, which complicates the task for the critic model within the

	GSM8K	MATH
Gemma-7B-Instruct	46.4	24.3
Gemma-2B-SFT	45.8	14.1
Gemma-2B-DPO	50.6	**16.0**
Gemma-2B-pDPO	**52.8**	15.7
DeepSeekMath-7B-Ins.	82.3	45.1
DeepSeekMath-7B-Ins. + DPO	**82.4**	46.3
DeepSeekMath-7B-Ins. + pDPO	82.3	**46.8**

Table 2: Experimental results on mathematical reasoning. *Ins.* is the short for *Instruct*, indicating we are using the instruction tuned version of DeepSeekMath. All experiments except the SFT one are repeated for **3 times** and the averaged results are reported.

PPO algorithm to accurately approximate the distribution and reduce the variance of the expected returns. Conversely, GRPO achieves a significant performance edge over PPO by sampling multiple solutions for the same query and calculating advantages using the group-averaged rewards as baseline. Furthermore, it is important to highlight that DPO-based methods significantly reduce training costs, completing the training process in under 16 hours on four NVIDIA H100 GPUs, whereas PPO and GRPO require over 40 hours on the same hardware.

5.3 Results on Mathematical Reasoning

In addition to logical reasoning, we also conducted experiments on mathematical reasoning to verify the effectiveness of our proposed approach, and the results are shown in Table 2. Specifically, we randomly sampled a subset of MetaMath (Yu et al., 2023) as the training set containing 25,000 questions for Gemma-2B training. From the table we can conclude that, on GSM8K, the synthesized process rewards also effectively enhance the mathematical reasoning capabilities. Moreover, by employing DPO and pDPO, our models with 2B parameters can outperform Gemma-7B-Instruct with significant improvements.

Despite our efforts, enhancing Gemma-2B-DPO's performance on the MATH dataset has proven challenging, possibly due to the base model's limited capability on MATH, which introduces noise when estimating expected returns during the simulation stage. Consequently, we expanded our experiments to include DeepSeekMath-7B-Instruct (Shao et al., 2024), which is pre-trained on large high-quality math-related corpus. We curated another subset from MetaMath for DeepSeekMath training, which contains 55,000 questions augmented from the MATH training dataset. As depicted in the table, the results reveal that pDPO also surpasses DPO in performance by employing better foundation model.

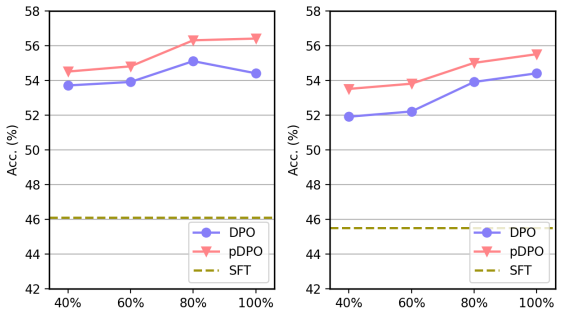


Figure 4: The accuracy of DPO, pDPO and SFT models on the validation set (left) and test set (right) of LogiQA-v2, respectively, taking different ratio of annotated questions.

5.4 Reliance on Annotations of Outcome Supervision

Although our proposed reward synthesis approach have avoided the direct annotation of process supervision, the outcome supervision still plays an important role for back-propagating the confidence to intermediate reasoning steps. In order to study the effect of outcome supervision scale to final performance, we randomly construct the sub-datasets containing 40%, 60%, and 80% questions in the original dataset to evaluate the fine-tuned performance. The results are plotted in Figure 4.

From the figure we can observe that (1) pDPO consistently outperform DPO across all dataset splits with different sizes by significant margins, demonstrating the effectiveness of the synthesized process supervision. (2) With only 40% annotations containing 3,234 questions in total, process supervision can outperform the base SFT model with significant improvements, which also verifies the significance by providing sparse feedback for continuous improvements. (3) Besides, we find that pDPO with only 40% outcome annotations can achieve comparable performance on the test set with DPO, i.e., 53.5 v.s. 53.9. Considering that we have only used **10%** outcome annotations for training the process reward model, the results can definitely emphasize the data efficiency of our approach.

5.5 Auto-evaluation of Rationale Quality by GPT-4

The most important concern is whether the synthesised process reward can contribute to reasonable and reliable rationale generation. In order to evaluate this, we propose to use GPT-4 for auto-

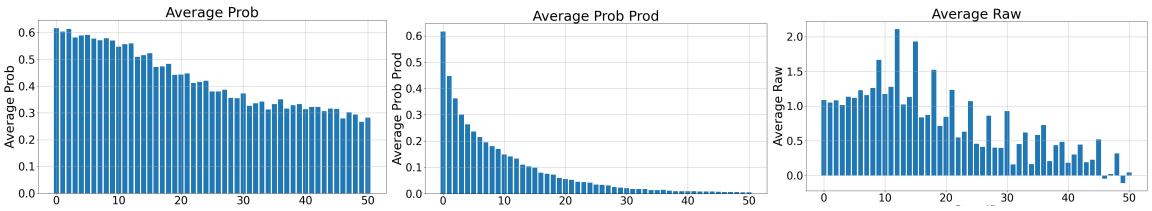


Figure 5: The averaged reward scores of intermediate reasoning steps predicted by our trained process-reward model on the training set of LogiQA-v2. The x-axis indicates the amount of reasoning steps and the y-axis describes the value of the averaged scores. For left to right, the three figures illustrate (1) predicted probability based reward of each reasoning step; (2) the accumulated probability based reward till specific reasoning step by production; and (3) the raw predicted reward values from the last layer of the reward model with different reasoning steps.

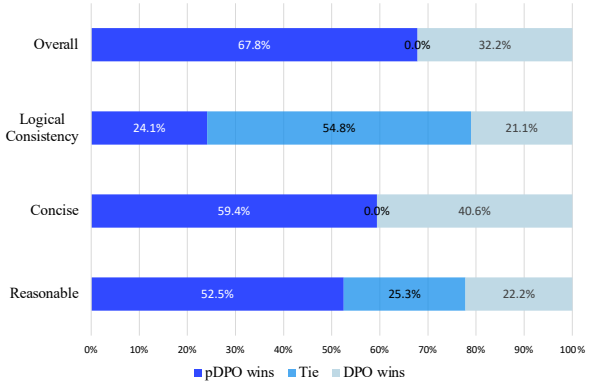


Figure 6: The wining rate between DPO and pDPO over different aspects of the auto-evaluation of GPT-4.

matic evaluation. Specifically, following Zhou et al. (2023a) and Zheng et al. (2023), we first formalize three dimensions to assess the rationales: *Reasonable*, *Concise*, and *Logically Consistent*. We then give GPT-4 two reasoning process and ask it to judge which one is better or it is a tie for each aspect. The critique details and prompt are shown in Figure 8. In order to avoid the bias caused by prediction errors of the two models, we first find a subset of questions where both the solutions given by the two models lead to the correct answer. After that, we randomly sampled 261 questions from the subset for evaluation. The results are shown in Figure 6. From all the three aspects, pDPO performs much better than vanilla DPO without process reward. Around 67.8% solutions of pDPO are deemed to have higher overall quality. Besides, for the most important view, among 52.5% questions, pDPO can generate more reasonable rationales. We can also find that nearly 60% responses by pDPO are more compact, suggesting that the process supervision can help make the rationale more brief but accurate.

5.6 Analysis of Predicted Rewards

In this section, we have visualized the predicted step-wise rewards on the training set of LogiQA-v2, where the solutions are sampled from the SFT model. In Figure 5, we have visualized three different kinds of rewards: (1) the averaged step-wise rewards before the softmax operation, i.e., the probability (left); (2) the accumulated rewards by production (medium); and (3) the averaged logits of each step from the last layer of the reward model (right). When diving into the logits without normalization, we can find that the rewards maintain relatively stable at around the first 15 steps, then decrease sharply. This may be caused by the imbalanced amount of solutions with different reasoning steps, which makes the reward model less confident on the longer steps. On the other hand, the accumulated probability based rewards keep decreasing with longer reasoning process, which can be useful to avoid redundant solutions by penalizing the extremely longer ones.

5.7 Case Study

In this section, we conduct a case study to intuitively demonstrate the augmentation bought by process-supervised DPO. As shown in Figure 9, the vanila DPO induced model shows two weaknesses: (1) the intermediate reasoning step is wrong, which is highlighted in red. And (2) the solution is redundant, like *Action 2* and *Action 5* to *Observation 8*. On the contrary, process-supervised DPO not only well illustrates the flaw in Q's response (*Observation 3*), but also eliminate the meaningless content, which introduce less noise to make correct prediction.

6 Conclusion

In this paper, we propose a novel idea to transform reasoning-as-planning as a learning problem to avoid the latency induced by online search. Inspired by MCTS, we developed a offline simulation approach to estimate the expected value of intermediate reasoning steps. After that, we use the collected expected value dataset to fit a process reward model and annotate the full trajectories with

sequence-level rewards. Finally, the policy model is optimized using direct preference optimization. The experimental results on logical and mathematical reasoning demonstrate the effectiveness of our proposed method. Towards the future work, we hope to explore the synthesised process reward estimated by weak-supervision from different aspects to further alleviate the reliance on human annotations and enable consistent self-improvement.

Limitations

The simulation based approach still requires large amount of resources, which has restricted some analysis for our approach, including experiments on competition level code generation that requires long context generation, and those taken on larger policy models.

Acknowledgements

This research is supported by the Ministry of Education, Singapore, under its Science of Learning Grant (Award ID MOE-MOESOL2021-0006). Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of the Ministry of Education, Singapore.

References

Mohammad Gheshlaghi Azar, Mark Rowland, Bilal Piot, Daniel Guo, Daniele Calandriello, Michal Valko, and Rémi Munos. 2023. A general theoretical paradigm to understand learning from human preferences. *CoRR*, abs/2310.12036.

Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, Binyuan Hui, Luo Ji, Mei Li, Junyang Lin, Runji Lin, Dayiheng Liu, Gao Liu, Chengqiang Lu, Keming Lu, Jianxin Ma, Rui Men, Xingzhang Ren, Xuancheng Ren, Chuanqi Tan, Sinan Tan, Jianhong Tu, Peng Wang, Shijie Wang, Wei Wang, Shengguang Wu, Benfeng Xu, Jin Xu, An Yang, Hao Yang, Jian Yang, Shusheng Yang, Yang Yao, Bowen Yu, Hongyi Yuan, Zheng Yuan, Jianwei Zhang, Xingxuan Zhang, Yichang Zhang, Zhenru Zhang, Chang Zhou, Jingren Zhou, Xiaohuan Zhou, and Tianhang Zhu. 2023. Qwen technical report. *arXiv preprint arXiv:2309.16609*.

Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, Carol Chen, Catherine Olsson, Christopher Olah, Danny Hernandez, Dawn Drain, Deep Ganguli, Dustin Li, Eli Tran-Johnson, Ethan Perez, Jamie Kerr, Jared Mueller, Jeffrey Ladish, Joshua Landau, Kamal Ndousse, Kamile Lukosiute, Liane Lovitt, Michael Sellitto, Nelson Elhage, Nicholas Schiefer, Noemí Mercado, Nova DasSarma, Robert Lasenby, Robin Larson, Sam Ringer, Scott Johnston, Shauna Kravec, Sheer El Showk, Stanislav Fort, Tamera Lanham, Timothy Telleen-Lawton, Tom Conerly, Tom Henighan, Tristan Hume, Samuel R. Bowman, Zac Hatfield-Dodds, Ben Mann, Dario Amodei, Nicholas Joseph, Sam McCandlish, Tom Brown, and Jared Kaplan. 2022. Constitutional AI: harmlessness from AI feedback. *CoRR*, abs/2212.08073.

Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, YinTat Lee, Yuanzhi Li, Scott Lundberg, Harsha Nori, Hamid Palangi, MarcoTulio Ribeiro, and Yi Zhang. 2023. Sparks of artificial general intelligence: Early experiments with gpt-4.

Paul F. Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. 2017. Deep reinforcement learning from human preferences. In *NeurIPS*, pages 4299–4307.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *CoRR*, abs/2110.14168.

Rémi Coulom. 2006. Efficient selectivity and backup operators in monte-carlo tree search. In *Computers and Games, 5th International Conference*, volume 4630 of *Lecture Notes in Computer Science*, pages 72–83. Springer.

Google DeepMind Gemma Team. 2024. Gemma: Open models based on gemini research and technology. *Preprint*, arXiv:2403.08295.

Shibo Hao, Yi Gu, Haodi Ma, Joshua Jiahua Hong, Zhen Wang, Daisy Zhe Wang, and Zhiting Hu. 2023. Reasoning with language model is planning with world model. *CoRR*, abs/2305.14992.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the math dataset. *NeurIPS*.

Geoffrey E. Hinton, Oriol Vinyals, and Jeffrey Dean. 2015. Distilling the knowledge in a neural network. *CoRR*, abs/1503.02531.

Jie Huang and Kevin Chen-Chuan Chang. 2023. Towards reasoning in large language models: A survey. In *Findings of ACL*, pages 1049–1065. ACL.

Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. 2023. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *CoRR*, abs/2311.05232.

Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lélio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2024. Mixtral of experts. *Preprint*, arXiv:2401.04088.

Fangkai Jiao, Zhiyang Teng, Shafiq R. Joty, Bosheng Ding, Aixin Sun, Zhengyuan Liu, and Nancy F. Chen. 2024. Exploring self-supervised logic-enhanced training for large language models. In *NAACL*. ACL.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.

Tamera Lanham, Anna Chen, Ansh Radhakrishnan, Benoit Steiner, Carson Denison, Danny Hernandez, Dustin Li, Esin Durmus, Evan Hubinger, Jackson Kernion, Kamile Lukosiute, Karina Nguyen, Newton Cheng, Nicholas Joseph, Nicholas Schiefer, Oliver Rausch, Robin Larson, Sam McCandlish, Sandipan Kundu, Saurav Kadavath, Shannon Yang, Thomas Henighan, Timothy Maxwell, Timothy Telleen-Lawton, Tristan Hume, Zac Hatfield-Dodds, Jared Kaplan, Jan Brauner, Samuel R. Bowman, and Ethan Perez. 2023. Measuring faithfulness in chain-of-thought reasoning. *CoRR*, abs/2307.13702.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023a. Let's verify step by step. *arXiv preprint arXiv:2305.20050*.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023b. Let's verify step by step. *CoRR*, abs/2305.20050.

Hanmeng Liu, Jian Liu, Leyang Cui, Nan Duan, Ming Zhou, and Yue Zhang. 2022. Logiqa2.0 dataset - logical reasoning in mrc and nli tasks. *TASLP*.

Haipeng Luo, Qingfeng Sun, Can Xu, Pu Zhao, Jianguang Lou, Chongyang Tao, Xiubo Geng, Qingwei Lin, Shifeng Chen, and Dongmei Zhang. 2023. Wizardmath: Empowering mathematical reasoning for large language models via reinforced evol-instruct. *CoRR*, abs/2308.09583.

OpenAI. 2023. Gpt-4 technical report. Technical report.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F. Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback. In *NeurIPS*.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. *CoRR*, abs/2305.18290.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *CoRR*, abs/1707.06347.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *CoRR*, abs/2402.03300.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*.

Avi Singh, John D. Co-Reyes, Rishabh Agarwal, Ankesh Anand, Piyush Patil, Xavier Garcia, Peter J. Liu, James Harrison, Jaehoon Lee, Kelvin Xu, Aaron Parisi, Abhishek Kumar, Alex Alemi, Alex Rizkowsky, Azade Nova, Ben Adlam, Bernd Bohnet, Gamaleldin F. Elsayed, Hanie Sedghi, Igor Mordatch, Isabelle Simpson, Izzeddin Gur, Jasper Snoek, Jeffrey Pennington, Jiri Hron, Kathleen Kenealy, Kevin Swersky, Kshiteej Mahajan, Laura Culp, Lechao Xiao, Maxwell L. Bileschi, Noah Constant, Roman Novak, Rosanne Liu, Tris Warkentin, Yundi Qian, Yamini Bansal, Ethan Dyer, Behnam Neyshabur, Jascha Sohl-Dickstein, and Noah Fiedel. 2023. Beyond human data: Scaling self-training for problem-solving with language models. *CoRR*, abs/2312.06585.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton-Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurélien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas

Scialom. 2023. Llama 2: Open foundation and fine-tuned chat models. *CoRR*, abs/2307.09288.

Jonathan Uesato, Nate Kushman, Ramana Kumar, H. Francis Song, Noah Y. Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. 2022. Solving math word problems with process- and outcome-based feedback. *CoRR*, abs/2211.14275.

Bin Wang, Zhengyuan Liu, Xin Huang, Fangkai Jiao, Yang Ding, Ai Ti Aw, and Nancy F. Chen. 2024. Seaeval for multilingual foundation models: From cross-lingual alignment to cultural reasoning. In *NAACL*. ACL.

Peiyi Wang, Lei Li, Zhihong Shao, R. X. Xu, Damai Dai, Yifei Li, Deli Chen, Y. Wu, and Zhifang Sui. 2023. Math-shepherd: Verify and reinforce llms step-by-step without human annotations. *CoRR*, abs/2312.08935.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS*.

Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, and Daxin Jiang. 2023. Wizardlm: Empowering large language models to follow complex instructions. *CoRR*, abs/2304.12244.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. 2023a. Tree of thoughts: Deliberate problem solving with large language models. *CoRR*, abs/2305.10601.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. 2023b. React: Synergizing reasoning and acting in language models. In *ICLR*. OpenReview.net.

Longhui Yu, Weisen Jiang, Han Shi, Jincheng Yu, Zhengying Liu, Yu Zhang, James T. Kwok, Zhenguo Li, Adrian Weller, and Weiyang Liu. 2023. Metamath: Bootstrap your own mathematical questions for large language models. *CoRR*, abs/2309.12284.

Ping Yu, Tianlu Wang, Olga Golovneva, Badr AlKhamissy, Gargi Ghosh, Mona T. Diab, and Asli Celikyilmaz. 2022. ALERT: adapting language models to reasoning tasks. *CoRR*, abs/2212.08286.

Weihao Yu, Zihang Jiang, Yanfei Dong, and Jiashi Feng. 2020. Reclor: A reading comprehension dataset requiring logical reasoning. In *ICLR*. OpenReview.

Zheng Yuan, Hongyi Yuan, Chengpeng Li, Guanting Dong, Chuanqi Tan, and Chang Zhou. 2023. Scaling relationship on learning mathematical reasoning with large language models. *CoRR*, abs/2308.01825.

Xiang Yue, Xingwei Qu, Ge Zhang, Yao Fu, Wenhao Huang, Huan Sun, Yu Su, and Wenhu Chen. 2023. Mammoth: Building math generalist models through hybrid instruction tuning. *CoRR*, abs/2309.05653.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging llm-as-a-judge with mt-bench and chatbot arena. *CoRR*, abs/2306.05685.

Chunting Zhou, Pengfei Liu, Puxin Xu, Srini Iyer, Jiao Sun, Yuning Mao, Xuezhe Ma, Avia Efrat, Ping Yu, Lili Yu, Susan Zhang, Gargi Ghosh, Mike Lewis, Luke Zettlemoyer, and Omer Levy. 2023a. LIMA: less is more for alignment. *CoRR*, abs/2305.11206.

Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc V. Le, and Ed H. Chi. 2023b. Least-to-most prompting enables complex reasoning in large language models. In *ICLR*. OpenReview.net.

A Baseline

Foundational LLMs We have selected the strong LLMs without task-specific fine-tuning as baselines, including Llama-2-70B-chat (Touvron et al., 2023), Mixtral-MoE-8×7B-Instruct (Jiang et al., 2024), GPT-3.5-Turbo and GPT-4-Turbo (OpenAI, 2023).

Supervised Fine-tuning (SFT) We first sample some responses from larger LLMs following the ReAct format for knowledge distillation since we cannot directly fine-tune them due to the resource limitation. After that, we can obtain the smaller LLMs with considerable reasoning capability through supervised fine-tuning (SFT). These models serve as baselines and the foundation models for DPO training. Specifically, we choose Llama-2-7B-chat and Gemma-2B-Instruct (Gemma Team, 2024) for SFT.

Outcome-based Preference Optimization We include the model with only outcome supervision as baseline to discuss the effectiveness of our synthesised process reward. For fair comparison, DPO implicitly model the outcome rewards following Equation 8. We also involve IPO as baseline. The training dataset is $\mathcal{D}_o$ as mentioned in Section 3.5.

Rejection Sampling-based Approach We also include the rejection sampling based approaches, i.e., Rejection sampling based Fine-tuning (Yuan et al., 2023), and ReST-EM (Singh et al., 2023). Both approaches use outcome annotations to filter the self-sampled solutions. The difference is that RFT uses the correct solutions to augment the original SFT dataset, while ReST-EM employs the sampled dataset to train the original model from scratch during each iteration. Besides, for RFT, we includes two variants: (1) RFT-outcome uses only the outcome annotation to filter solutions; and (2) RFT-outcome & PRM-top-k follows RFT-outcome and uses our trained PRM to rank the kept solutions. Only the top-k ranked solutions will be kept and augment the orinal training set. For ReST-EM, we have conducted two iterations since there is already performance decreasing observed in the second round.

Reinforce Learning In the experiments of iterative training, we include two reinforce learning algorithms, PPO (Schulman et al., 2017) and GRPO (Shao et al., 2024) as the comparison of process-based DPO. Both algorithms employ two kinds of rewards, i.e., the outcome reward and the process rewards. For each solution (trajectory) sampled from the policy model, we assign it with 1 if it can induce the correct answer, otherwise we assign it with 0 as the outcome reward. Besides, for each reasoning step, the predicted logits by our trained PRM is treated as the process rewards. One difference should be noted is that, in pDPO training, we utilize the probability from the PRM as the process reward following Lightman et al. (2023a), while for RL training, we use the logits without normalization from the last layer of PRM, to avoid extreme longer solutions introduced by accumulating the non-positive step rewards.

B Evaluation Details

In order to simplify the evaluation procedure, for the models without task-specific fine-tuning, we use 1-shot prompt of ReAct, which is the same as that we used for collecting data, to induce the models to generate reasonable solutions. For models after fine-tuning, we remove the 1-shot demonstration because we find it can lead to higher results. Due to limitation of budget, for GPT-4-Turbo, we only evaluate the first 250 questions in the test set of LogiQA-v2.

Besides, as mentioned in Section 4.3, we have designed several rules to both filter the solutions unsatisfying the ReAct format and calculate the accuracy. Specifically, all of the following cases will be considered incorrect:

- The final answer contains more than one prediction, e.g., *Finish[The answer is A and B]*.
- The solution is truncated due to the length limit, but some option indices are gathered.
- The summary format is incorrect, e.g., *Finish: the answer is A*.

For experiments with DeepSeekMath (Shao et al., 2024) on mathematical reasoning, we only do basic cleaning like removing the redundant newline symbols, since it is already fine-tuned on the solutions with CoT format.

C Implementation Details

C.1 Data Preparation

Considering the limited computation resources, we mainly conducted experiments on Llama2-7b-chat (Touvron et al., 2023), DeepSeek-

Dataset	# Question (Train)	Avg. of Correct Solutions Per. Question	# Question (Val.)	# Question (Test)
LogiQA-v2	12,567	6.0	1,569	1,572
ReClor	4,638	5.0	500	1,000

Table 3: Statistics of our used datasets in this paper for construction preference pairs. The solutions shown in the table are sampled from the corresponding SFT model based on the questions in the training set.

σ	No. of Pairs	No. of P. Pairs	Ratio of P. Pairs	Dev.	Test
1.0	133,458	0	0	54.4	54.4
0.3	179,776	46,318	25.8%	51.4	50.4
0.5	161,140	27,682	17.2%	**56.4**	**55.5**
0.7	148,136	14,678	9.9%	55.7	54.3

Table 4: Accuracy on LogiQA-v2 dataset with different σ. $\sigma = 1.0$ refers to the vanilla DPO method. *P. Pairs* refers to *process-supervised sample pairs*.

Math (Shao et al., 2024)-7B-Instruct, and Gemma-2B-Instruct (Gemma Team, 2024). In order to collect solutions reaching correct answers more efficiently, we first fine-tune the original models on corresponding dataset using the generated responses from some teacher models (except DeepSeekMath since its solutions are already in CoT format). For LogiQA-v2, we sample solutions from Llama-2-70b-chat, while for ReClor, the solutions are sampled from GPT-3.5-Turbo to save time. For Gemma-2B, we sample solutions of MetaMath from Qwen-72B-chat (Bai et al., 2023).

All teacher models are prompted with exactly one example. The prompt used for LogiQA-v2 and ReClor is shown in Figure 7. And the one used for MetaMath follows RAP (Hao et al., 2023)[4]. For all datasets, we sample 10 solutions regarding each question with temperature fixed as 0.7. Besides, for ReClor dataset, we remove all solutions with less than 8 reasoning steps because they omit the detailed reasoning process and can lead to inferior solutions for DPO based approach.

C.1.1 Training Data Collection For PRM

For LogiQA-v2, we randomly sampled 10% questions from the training set for process rewards estimation and PRM training. For ReClor, the ratio is 20%. For Gemma-2B training, we have used 25% questions for PRM training, while for DeepSeek-Math, we have used around 10%.

C.2 Hyper-Parameters

For hyper-parameters, we use $\beta = 0.1$ and $C = 2$ on logical reasoning tasks, and $\beta = 0.5$, $C = 3$ on mathematical reasoning tasks. Besides, σ is set as 0.4 for ReClor dataset, 0.5 for LogiQA-v2, 0.5 for Gemma-2B, and 0.3 for DeepSeekMath.

C.3 Training

All experiments are conducted on NVIDIA A100 and H100. The evaluation of LLMs relies on vLLM (Kwon et al., 2023) inference backend. For logical reasoning, after training, we evaluate all checkpoints on the development set of the target dataset using greedy decoding, and select the best one to report its performance on the test set. For Gemma-2B, we select the model checkpoint based on the performance on GSM8K, and for DeepSeek-Math, we report the performance of the best checkpoint on MATH. All experiments, expept those using RL algorithms, are **repeated for 3 times** with different random seeds and the average results are reported to reduce the influence of randomness. We run RL-based approaches for only once due to resource limitation.

[4] https://github.com/Ber666/RAP/data/gsm8k/prompts.

D Compared with MATH-Shepherd

We work concurrently with Math-Shepherd (Wang et al., 2023), which also comprises similar offline simulation method to synthesize the process supervision. Differently, they mainly evaluate the approach on mathematical reasoning through verification, where the candidate solutions are ranked according to the rewards from the learned PRM, or employing it for PPO training, while we focus on logical reasoning and demonstrate the effectiveness of the synthesized process supervision via constructing the preference dataset under the guidance of the PRM. The dataset is further used for DPO training, which, though cannot really surpass GRPO, often demonstrates less resource requirements and more stable learning process.

E Effect of Different Reward Margins

In Equation 9, we have involved a hyper-parameter σ to control the confidence interval between different sample pairs both reaching the correct answer to construct the process-supervised preference dataset. Naturally, there are several aspects of trade-off to considering the choices of σ. σ with higher value can improve the ratio of true positive pairs in the constructed dataset. Yet, high confidence intervals will also reduce the number

of training data and probability to include more hard negative samples. For example, as shown in Table 4, $\sigma = 0.7$ introduces only 10% extra preference pairs and lead to less significant improvements compared with the case where $\sigma = 0.5$. On the other hand, lower value of σ can include both more hard negative and false positive pairs. From the table we find that $\sigma = 0.3$ has has introduced more than 25% process-supervised pairs, but the performance is even worse than the vanilla DPO approach, where only outcome-based preferences pairs are employed.

Solve a question answering task by having a Thought, then Finish with your answer. Thought can reason about the current situation. Finish[answer] returns the answer and finishes the task. You will be given context that you should use to help you answer the question.

Context:
A college will continue to implement the overseas funding plan this year. It plans to select several of the six teachers from Mr. Liu, Mr. Zhang, Mr. Wang, Mr. Ma, Mr. Niu and Mr. Zhou to visit abroad. Due to the limitations of funding, the needs of discipline development, curriculum arrangement, place and time of each student's visit, the selection shall meet the following conditions: (1) Mr. Liu is the reserve discipline leader of the college, This time we have to send out. (2) if we choose Mr. Liu, we should also choose Mr. Zhou, but we can't choose Mr. Zhang. (3) only if Mr. Niu can't choose, at least one of Mr. Wang and Mr. Ma can choose. (4) if we don't choose Mr. Wang, we don't choose Mr. Zhou either.

Question: If the above statement is true, which of the followings must be true?

Options:
A. Mr. Niu didn't choose, but Mr. Zhou did.
B. Mr. Liu was chose, but Mr. Ma didn't.
C. Mr. Wang and Mr. Ma were chosen.
D. Neither Mr. Wang nor Mr. Niu was elected.

Thought 1: Who must be chosen or who must not be chosen according to the context by one-step reasoning?
Action 1: Take condition (1).
Observation 1: Mr. Liu must be chosen.
Thought 2: Based on Observation 1, who must be chosen or who must not be chosen one step further?
Action 2: Take condition (2).
Observation 2: Mr. Zhou must be chosen, but Mr. Zhang must not be chosen.
Thought 3: Based on Observation 2, who must be chosen one step further? And who must not be chosen?
Action 3: No further conditions can be taken to derive new observations.
Thought 4: We can evaluate each option to check if there is any contradiction with the observations.
Action 4: Take option A.
Observation 4: Mr. Niu didn't choose, but Mr. Zhou did.
Thought 5: Which observations and conditions are related to Option A?
Action 5: Observation 2 and condition (3).
Observation 5: We can confirm that Mr. Zhou was chosen according to observation 2. Condition (3) shows that if Mr. Niu was not chosen, at least one of Mr. Wang and Mr. Ma must be chosen. This needs further analysis by assumption.
Thought 6: If Mr. Niu was not chosen is true, what will happen? We should examine the truth of the consequences.
Action 6: Assume that Mr. Niu was not chosen.
Observation 6: At least one of Mr. Wang and Mr. Ma must be chosen.
Thought 7: Which observations and conditions are related to Observation 6 to check if there is any contradiction?
Action 7: Take condition (4).
Observation 7: If Mr. Wang was not chosen, Mr. Zhou must not be chosen. This contradicts Observation 2. Therefore, Mr. Wang must be chosen. So Observation 6 is true. So Option A is true.
Action 8: Take option B.
Observation 8: Mr. Liu was chosen, but Mr. Ma didn't.
Thought 9: Which observations and conditions are related to Option B?
Action 9: Observation 1 and condition (3).
Observation 9: Mr. Liu must be chosen. But we cannot tell whether Mr. Ma was chosen or not. So Option B is not necessarily true.
Action 10: Take option C.
Observation 10: Mr. Wang and Mr. Ma were chosen.
Thought 11: Which observations and conditions are related to Option C?
Action 11: Condition (3) and condition (4).
Observation 11: We cannot tell whether Mr. Wang and Mr. Ma were chosen or not. So Option C is not necessarily true.
Action 12: Take option D.
Observation 12: Neither Mr. Wang nor Mr. Niu was elected.
Thought 13: Which observations and conditions are related to Option D?
Action 13: Condition (3) and condition (4).
Observation 13: According to condition (3), if Mr. Niu was not chosen, at least one of Mr. Wang and Mr. Ma must be chosen. But Mr. Wang was not chosen according to observation 10. So Mr. Ma must be chosen. According to condition (4), if Mr. Wang was not chosen, Mr. Zhou must not be chosen. So Mr. Zhou was not chosen. This is contradictory to observation 2. So Option D is not true.
Action 14: Finish[The answer is A.]

Context:
[New Context]

Question: [New Question]

Options:
[New Options]

Figure 7: Prompt for sampling ReAct solutions. For zero-shot prompting, the exemplar is removed while the other content keep unchanged.

Here is a logical reasoning problem, and there are two solutions describing their thinking process. Please tell me which one is better. You should consider the following criteria:
1. Reasonable: The reasoning process is reasonable, which means each conclusion should be inferred by collecting evidence, instead of making up unknown facts.
2. Concise: The reasoning process should not tell something irrelevant to the question.
3. Logically consistent: There must not be contradiction in the process itself.

Here is the problem containing a context, a question, and different options:

Context:
[Context]

Question:
[Question]

Options:
[Options]

Reasoning process A:
[Reasoning process A]

Reasoning process B:
[Reasoning process B]

For each aspect of the above criteria, select one winner, or judge it as a tie, following the format:
Reasonable: A/B/Tie
Concise: A/B/Tie
Logically consistent: A/B/Tie

And finally, by considering all the criteria together, select one winner, or judge it as a tie, with the following format:
Overall: A/B/Tie

Figure 8: Prompt for compare two different responses, and assess the quality from different aspects.

Context:
P: Because an elected official needs the support of a political party to be effective, the independent candidate for the legislature cannot possibly be an effective legislator is she wins. Q: I disagree. By your reasoning, our current legislator, who has the support of a political party, ought to have been effective, but he has not been.

Question:
Which one of the following is the best criticism of Q's statement?

Options:
A. It mistakenly interprets P to be claiming that a factor assures, rather than is necessary for, a legislator's effectiveness.
B. It simply contradicts P's claim without offering evidence against it.
C. It does not consider the possibility that a political party might decide to support an elected legislator even though he or she ran as an independent.
D. It fails to prove a precise definition for a key term -- the word effective.

[Llama-2-7b-DPO]:
Thought 1: What is the main point of P's statement?
Action 1: Read P's statement.
Observation 1: P says that an elected official needs the support of a political party to be effective, and an independent candidate for the legislature cannot possibly be an effective legislator if she wins.
Thought 2: What is Q's statement?
Action 2: Read Q's statement.
Observation 2: Q says that by P's reasoning, the current legislator, who has the support of a political party, ought to have been effective, but he has not been.
Thought 3: What is the relationship between P's and Q's statements?
Action 3: Analyze the relationship.
Observation 3: Q is challenging P's claim that a political party is necessary for a legislator's effectiveness.
Thought 4: Which option best criticizes Q's statement?
Action 4: Evaluate each option.
Observation 4: Option A is incorrect because Q does not mistakenly interpret P's claim. Option B is incorrect because Q does not simply contradict P's claim without offering evidence against it. Option C is incorrect because Q does not consider the possibility that a political party might decide to support an elected legislator even though he or she ran as an independent. Option D is incorrect because Q does not fail to prove a precise definition for a key term.
Action 5: Take option A.
Observation 5: Option A is incorrect.
Action 6: Take option B.
Observation 6: Option B is incorrect.
Action 7: Take option C.
Observation 7: Option C is incorrect.
Action 8: Take option D.
Observation 8: Option D is incorrect.
Action 9: Finish[The answer is A.]

[Llama-2-7b-pDPO]:
Thought 1: What is the main point of P's statement?
Action 1: Analyze the premises and conclusion of P's statement.
Observation 1: P argues that an elected official needs the support of a political party to be effective, and therefore, an independent candidate for the legislature cannot be an effective legislator if she wins.
Thought 2: What is Q's response to P's statement?
Action 2: Analyze Q's statement.
Observation 2: Q disagrees with P's argument by pointing out that the current legislator, who has the support of a political party, has not been effective despite having the support of a political party.
Thought 3: What is the flaw in Q's response?
Action 3: Evaluate Q's response based on P's argument.
Observation 3: Q's response does not address the main point of P's statement. Q simply provides a counterexample to P's argument without acknowledging or refuting P's premises.
Thought 4: Which option best describes the flaw in Q's response?
Action 4: Evaluate each option.
Option A: Acknowledges the mistake in Q's response. Q does not mistakenly interpret P's claim. Q simply ignores P's premises.
Option B: Does not accurately describe the flaw in Q's response. Q does not simply contradict P's claim. Q provides a counterexample to P's argument.
Option C: Not relevant to the flaw in Q's response. Q does not consider the possibility that a political party might decide to support an elected legislator even though he or she ran as an independent.
Option D: Not relevant to the flaw in Q's response. Q does not fail to prove a precise definition for a key term.
Action 5: Finish[The answer is A.]

Figure 9: Case study taking the responses from Llama2-7b-DPO and Llama2-7b-pDPO that are fine-tuned on LogiQA-v2, respectively.